

Coachie AI

최종 보고서

AI 기반 목표 설정과 식단 기록을 결합한 Android 모바일 앱

앱 이름	Coachie AI
개요	내가 목표하는 몸을 위해, AI의 도움을 받아 목표를 세우고, 먹는 음식들을 기록하는 앱
개발 환경	Kotlin, Android Studio, Jetpack Compose, Room, Firebase Auth, Cloud Firestore, Firebase AI Gemini
주요 기능	AI 식단 분석, 목표 기반 영양 대시보드, 신체 측정 기록, 사용자별 Firebase 동기화, 식사 기록 알림

최종 보고서 목차

1. 프로젝트 개요 및 기획 의도

- 개발 배경
- 앱이 제공하는 핵심 가치

2. 시스템 아키텍처 및 기술 스택 선정 이유

- 전체 시스템 구조
- 기술 스택별 선정 이유

3. 핵심 기능 구현 세부 사항

- AI 식단 분석 및 리뷰 후 저장
- 목표 계획과 신체 측정값 관리
- 사용자별 Firebase 데이터 동기화
- UI/UX 디자인 콘셉트 및 주요 화면 캡처

4. 트러블 슈팅 및 문제 해결

- Firebase 로그인 실패 문제
- 로그인 후 데이터가 비어 보이는 문제
- AI 분석 결과 신뢰도 문제
- 목표 데이터 구조 변경 문제

5. 프로젝트 후기 및 향후 발전 방향

- 프로젝트를 통해 배우고 성장한 점
- 시간 관계상 구현하지 못해 향후 추가하고 싶은 기능

6. 프로젝트 요구사항 만족여부 체크리스트

1. 프로젝트 개요 및 기획 의도

1.1 개발 배경

다이어트와 벌크업은 단순히 의지만으로 유지하기 어렵다. 사용자는 매일 어떤 음식을 먹었는지, 하루 목표 칼로리와 탄수화물, 단백질, 지방 목표에 얼마나 가까운지 확인해야 한다. 하지만 기존의 식단 기록 방식은 매 끼니마다 음식명, 칼로리, 영양소를 직접 찾아 입력해야 하므로 번거롭다. 이런 입력 부담은 사용자가 기록을 중단하게 만드는 가장 큰 원인 중 하나다.

Coachie AI는 이 문제를 줄이기 위해 기획한 앱이다. 사용자는 음식 사진과 간단한 설명을 입력하고, AI가 추정한 영양 정보를 확인한 뒤 저장할 수 있다. 또한 현재 체중, 목표 체중, 골격근량, 체지방률 같은 신체 정보를 기반으로 목표 계획을 세우고, 실제 섭취량이 목표와 얼마나 맞는지 홈 화면과 통계 화면에서 확인할 수 있다.

1.2 앱이 제공하는 핵심 가치

핵심 가치	설명
기록 부담 감소	음식 사진과 설명을 기반으로 AI가 음식명과 영양 정보를 추정한다. 사용자는 모든 값을 직접 입력하는 대신 AI 결과를 확인하고 필요한 부분만 수정하면 된다.
목표 기반 피드백	먹은 음식을 단순히 저장하는 데서 끝나지 않고, 하루 목표와 비교해 부족하거나 초과한 영양 정보를 보여준다. 사용자는 다음 식사를 더 쉽게 조절할 수 있다.
사용자별 데이터 관리	Firebase Auth 계정과 로컬 Room 데이터를 연결하고, Firestore에 사용자별 스냅샷을 저장한다. 발표용 테스트 계정처럼 여러 사용자의 데이터를 분리하여 관리할 수 있다.

초기 아이디어는 단순한 식단 기록 앱이었지만, 실제 사용 흐름을 생각하면서 목표 설정, 신체 기록, AI 분석, 알림, Firebase 동기화 기능까지 확장되었다. 결과적으로 Coachie AI는 사용자가 자신의 몸 목표를 중심으로 식단을 기록하고 점검하는 개인 코치형 앱을 목표로 한다.

2. 시스템 아키텍처 및 기술 스택 선정 이유

2.1 전체 시스템 구조

앱은 Compose UI, Navigation, Repository, Room Database, Firebase 계층으로 나누어 구성했다. 화면은 직접 DB나 Firebase에 접근하지 않고 Repository를 통해 데이터를 읽고 저장한다. 이 구조는 화면 전환이 많고 데이터 흐름이 복잡한 앱에서 기능을 분리하기 쉽다.

사용자

- > Jetpack Compose 화면
- > AIDietNavHost
- > Repository Layer
- > Room Database / Firebase Auth / Cloud Firestore / Firebase AI

2.2 기술 스택 선정 이유

기술	사용 위치	선정 이유
Kotlin	Android 앱 전체	Android 공식 권장 언어이며, data class, coroutine, null safety를 통해 데이터 중심 앱을 안정적으로 구현할 수 있다.
Jetpack Compose / Material3	전체 UI	선언형 UI 구조라 홈 대시보드, 목표 설정, 리뷰 화면처럼 상태 변화가 많은 화면을 빠르게 구성할 수 있다.
Navigation Compose	화면 전환	Login, Home, Add Meal, Review, Detail, Goal, Profile 등 여러 화면을 route 기반으로 연결하기 위해 사용했다.
Room / Flow	로컬 데이터 저장	오프라인에서도 식단, 목표, 신체 기록을 볼 수 있어야 하며, Flow로 DB 변경을 UI에 자동 반영할 수 있다.
Firebase Auth / Firestore / AI	인증, 클라우드 데이터, AI 분석	사용자별 계정 관리, 발표용 데이터 동기화, Gemini 기반 식단 분석을 구현하기 위해 사용했다.
AlarmManager	식사 기록 알림	아침, 점심, 저녁 식사 기록 습관을 돕기 위해 로컬 알림을 구현했다.

3. 핵심 기능 구현 세부 사항

3.1 핵심 기능 1: AI 식단 분석 및 리뷰 후 저장

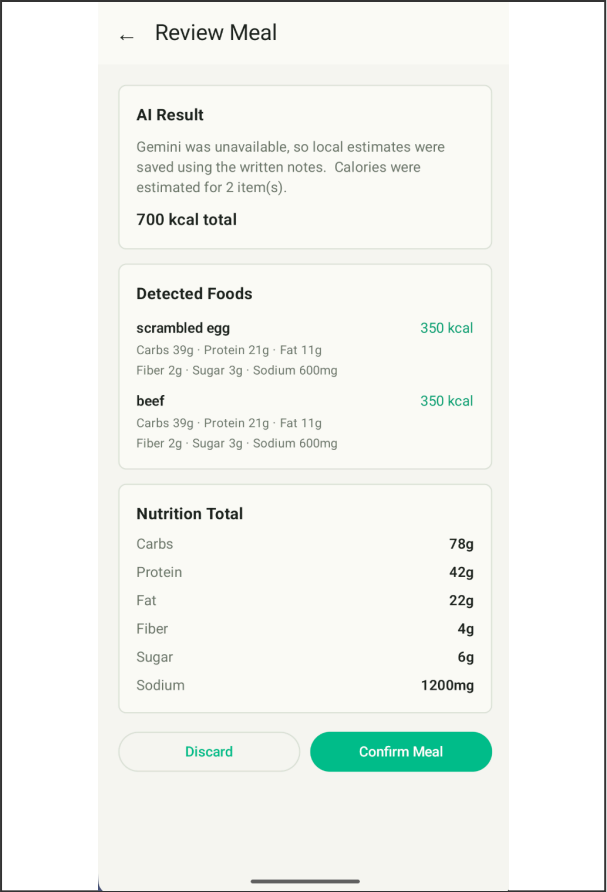
식단 기록은 단순 입력 폼으로 끝나지 않는다. 사용자가 음식 사진과 설명을 입력하면 MealUploadDraft가 만들어지고, MealReviewRepository가 Firebase AI Gemini 또는 로컬 추정 로직을 통해 MealRecord를 생성한다. 이후 바로 저장하지 않고 리뷰 화면에서 결과를 확인한 뒤 확정하도록 설계했다.

```
val startMealAnalysis: (MealUploadDraft) -> Unit = { mealUploadDraft ->
    val draft = mealUploadDraft.copy(ownerId = currentUserId)
    pendingMealReview = PendingMealReviewState(
        draft = draft,
        isAnalyzing = true,
        reviewedMeal = null,
        errorMessage = null
    )

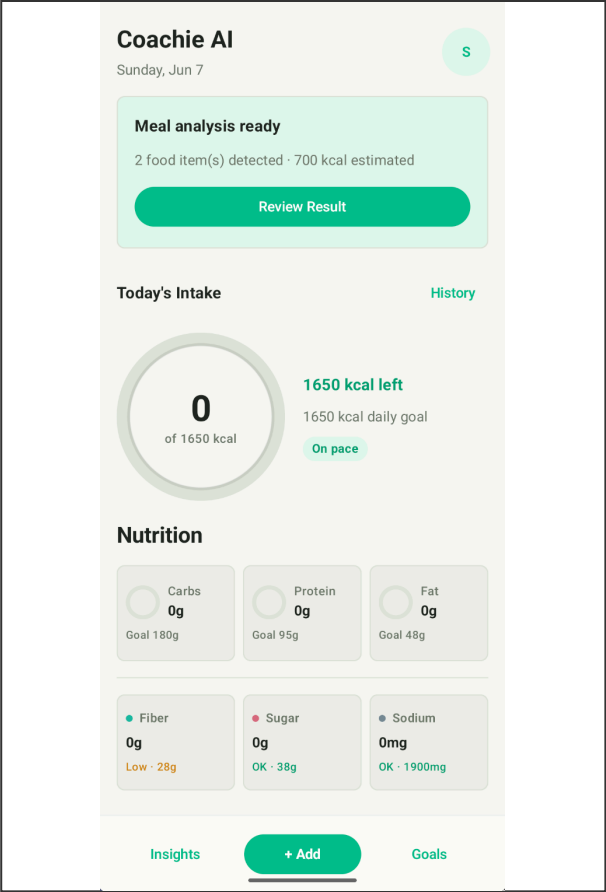
    coroutineScope.launch {
        val reviewedMeal = mealReviewRepository.reviewMealDraft(draft)
        pendingMealReview = pendingMealReview?.copy(
            isAnalyzing = false,
            reviewedMeal = reviewedMeal,
            errorMessage = null
        )
    }
}
```

이 흐름의 장점은 사용자가 AI 추정값을 무조건 신뢰하지 않아도 된다는 점이다. AI가 추정한 칼로리와 영양소는 리뷰 화면과 수정 화면에서 확인할 수 있고, edit 모드에서는 직접 수정할 수 있다. 따라서 AI는 입력을 돕는 보조자 역할을 하고, 최종 저장 권한은 사용자에게 남겨두었다.

관련 화면



AI 분석 결과 리뷰 화면



분석 완료 후 홈 화면

3. 핵심 기능 구현 세부 사항

3.2 핵심 기능 2: 목표 계획과 신체 측정값 관리

Coachie AI의 목표 기능은 단순히 목표 칼로리 하나를 저장하는 방식이 아니다. 사용자는 현재 체중, 목표 체중, 골격근량, 체지방률, 기초대사량, 목표 기간을 입력하고, 앱은 이를 바탕으로 하루 영양 목표를 계산한다. 목표는 `goal_plans` 테이블에 기간 기반으로 저장되고, 신체 측정값은 `body_measurements` 테이블에 날짜별로 저장된다.

```
suspend fun saveGoalPlan(ownerId: Long, draft: GoalPlanDraft) {
    val validToEpochDay = draft.validToEpochDay
        .takeIf { it > draft.validFromEpochDay }
        ?: (draft.validFromEpochDay + 1L)

    mealDatabase.withTransaction {
        goalDao.closeOverlappingGoalPlans(ownerId, draft.validFromEpochDay)
        goalDao.deleteGoalPlansStartingAtOrAfter(ownerId, draft.validFromEpochDay)
        goalDao.insertGoalPlan(goalPlan)
        goalDao.insertBodyMeasurement(startMeasurement)
    }
}
```

목표를 기간 기반으로 저장한 이유는 식단 통계와 연결하기 위해서다. 예를 들어 사용자가 중간에 벌크업 목표에서 감량 목표로 바꾸면, 과거 기록은 이전 목표와 비교되어야 하고 이후 기록은 새 목표와 비교되어야 한다. 이를 위해 목표를 덮어쓰기보다 적용 시작일과 종료일을 가진 이력 데이터로 설계했다.

관련 화면

← Goal Settings

Set a required body weight, optional body composition, and target values. The app will build a daily nutrition plan from them.

Goal Period

Target Date

Duration

Start Date

Target Date

2026-05-12

2026-07-06

Plan period: 2026-05-12 ~ 2026-07-06 · 56 days (8 weeks)

Current Profile

Latest body status: 2026-06-05 · 56.5kg · 1320kcal
BMR · 22.2kg muscle · 14.9kg fat · 26.4% fat

Use Latest Body Status

Hide Body Status Input

Measured Date

2026-06-07

Weight

kg

Metabolic Rate

Skeletal Muscle

Body Fat Mass

목표 설정 화면

← Goal Settings

Current Profile

Latest body status: 2026-06-05 · 56.5kg · 1320kcal
BMR · 22.2kg muscle · 14.9kg fat · 26.4% fat

Use Latest Body Status

Hide Body Status Input

Measured Date

2026-06-07

Weight

kg

Metabolic Rate

Skeletal Muscle

Body Fat Mass

Body Fat %

Only one value is required. Weight and body fat mass can estimate body fat percent for the AI plan.

Cancel

Save Body Status

Current Weight *

58.4

kg

Skeletal Muscle

22.1

kg

Body Fat %

28.6

%

Metabolic Rate

목표 및 신체 기록 입력

3. 핵심 기능 구현 세부 사항

3.3 핵심 기능 3: 사용자별 Firebase 데이터 동기화

Firebase Auth로 로그인한 사용자는 고유한 Firebase UID를 가진다. 앱은 이 UID를 Room의 로컬 사용자와 연결하고, 사용자의 식단, 목표, 신체 측정 데이터를 Firestore의 `users/{firebaseUid}/data/current` 문서에 저장한다. 로컬에서 데이터를 추가, 수정, 삭제하면 현재 Room 상태를 스냅샷으로 업로드하고, 로그인 시에는 Firestore 스냅샷을 Room으로 내려받는다.

```
suspend fun downloadUserDataSnapshot(userAccount: UserAccount):
    FirebaseUploadSummary? {
    val snapshot = firestore.collection("users")
        .document(userAccount.firebaseUid!!)
        .collection("data")
        .document("current")
        .get()
        .await()

    if (!snapshot.exists()) return null

    mealDatabase.withTransaction {
        mealDao.deleteMealsForOwner(userAccount.id)
        goalDao.deleteGoalPlansForOwner(userAccount.id)
        goalDao.deleteBodyMeasurementsForOwner(userAccount.id)

        // Firestore meals, goals, body logs are inserted into Room.
    }
}
```

처음에는 export/import JSON만 제공했지만, 발표용 테스트 계정 데이터를 계정별로 유지하려면 클라우드 스냅샷이 필요했다. 그래서 기존 export/import 기능은 그대로 유지하면서 Firestore 동기화 레이어를 추가했다. 로그인 직후 데이터가 비어 보이지 않도록, 앱 시작 시 기존 Firebase 세션이 복원되는 경우에도 같은 다운로드 로직을 실행한다.

동기화 흐름

상황	동작
식단 추가/수정/삭제	Room DB 변경 후 현재 사용자 스냅샷을 Firestore에 업로드
목표 또는 신체 측정값 변경	Room DB 저장 후 사용자별 Firestore 문서 갱신
로그인 또는 세션 복원	Firestore의 현재 스냅샷을 Room DB로 import

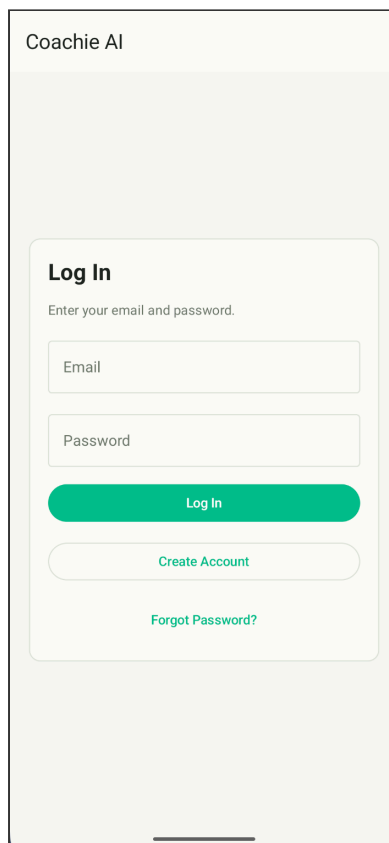
JSON import	기존 export/import 기능으로 Room 복원 후 Firestore 업로드
-------------	---

3. 핵심 기능 구현 세부 사항

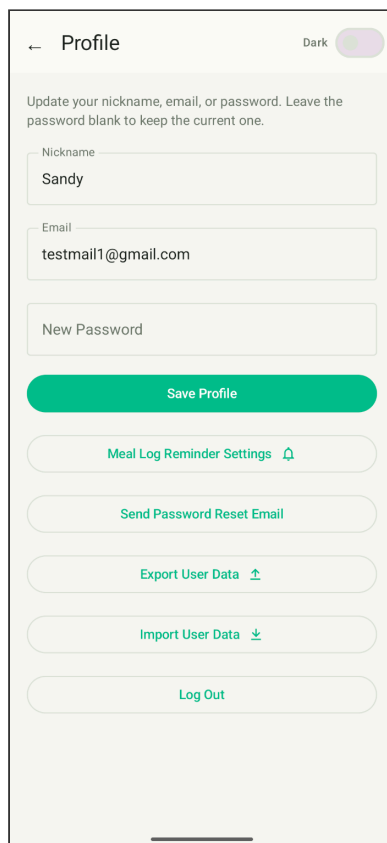
3.4 UI/UX 디자인 콘셉트 및 주요 화면

UI는 운동 기록 앱보다는 식단과 건강 관리를 함께 다루는 웰니스 앱처럼 보이도록 설계했다. 배경은 밝은 그린 톤의 저채도 색을 사용하고, key color는 #4DBA8C로 통일했다. 앱 아이콘도 같은 key color를 바탕으로 단순한 체크 로고 형태로 제작했다.

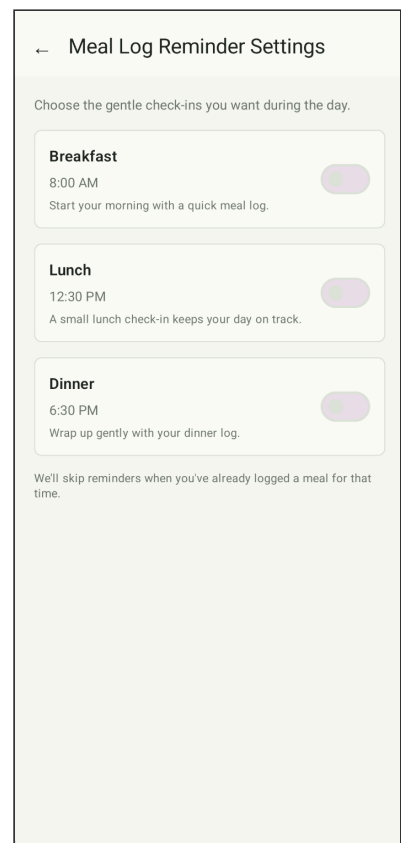
홈 화면은 사용자가 가장 자주 보는 화면이므로 큰 칼로리 게이지와 매크로 요약을 중심으로 구성했다. 목표 화면은 처음부터 복잡한 입력 폼을 보여주기보다 현재 목표 요약을 먼저 보여주고, 필요할 때 편집하도록 흐름을 바꾸었다. 프로필 화면에는 테마 전환, 알림 설정, export/import를 모아 데이터 관리의 중심 화면으로 구성했다.



로그인 화면



프로필 및 데이터 관리



식사 기록 알림 설정

README에는 사용자가 직접 생성한 GIF 파일을 연결하여 홈 화면 흐름과 다크 모드 동작을 확인할 수 있도록 했다.

4. 트러블 슈팅 및 문제 해결

4.1 Firebase 로그인 실패

에뮬레이터에서 Firebase 로그인이 계속 실패하면서 앱에는 "Could not reach Firebase" 메시지가 표시되었다. 처음에는 Firebase 설정이나 계정 문제가 원인이라고 생각했지만, 같은 API key와 계정 정보를 REST API로 테스트했을 때 로그인 자체는 성공했다. 따라서 문제는 앱 코드가 아니라 에뮬레이터 네트워크 환경에 있다고 판단했다.

최종적으로 에뮬레이터를 다음 옵션으로 직접 실행하여 해결했다.

```
~/Library/Android/sdk/emulator/emulator \  
-avd Medium_Phone35 \  
-dns-server 8.8.8.8,1.1.1.1 \  
-no-snapshot-load
```

4.2 로그인 후 데이터가 비어 보이는 문제

Sandy 계정의 발표용 데이터를 Firestore에 업로드했지만, 앱에서 로그인하면 아무 기록도 보이지 않았다. 원인은 앱이 Room DB를 기준으로 화면을 그리는데, 당시 구현은 로컬 변경 사항을 Firebase에 업로드하는 기능만 있고 Firebase 데이터를 Room으로 내려받는 기능은 없었기 때문이다.

이를 해결하기 위해 `FirebaseUserDataRepository.downloadUserDataSnapshot()`을 추가하고, 로그인 성공 시와 앱 시작 후 기존 Firebase 세션 복원 시 이 함수를 호출하도록 했다.

4.3 AI 분석 결과의 신뢰도 문제

AI 음식 분석은 편리하지만 사진 각도, 음식량, 조리 방식에 따라 결과가 달라질 수 있다. AI 결과를 자동 저장하면 사용자가 잘못된 값을 그대로 누적할 위험이 있다. 그래서 Coachie AI는 AI 분석을 최종 판단이 아니라 입력 보조로 설계했다. 사용자는 리뷰 화면에서 확인하고 edit 화면에서 직접 수정할 수 있다.

4.4 목표 데이터 구조 변경

초기 목표 설정은 화면 상태에 가까웠다. 하지만 실제로 앱을 사용한다고 생각해보면 목표는 시간이 지나며 바뀌고, 과거 기록은 과거 목표와 비교되어야 한다. 이를 해결하기 위해 목표를 GoalPlanEntity로 분리하고, 적용 시작일과 종료일을 가진 기간 데이터로 저장했다.

5. 프로젝트 후기 및 향후 발전 방향

5.1 프로젝트를 통해 배우고 성장한 점

이번 프로젝트를 통해 단순히 기능을 많이 넣는 것보다, 사용자가 어떤 순서로 앱을 사용해야 편한지 계속 생각하는 일이 중요하다는 것을 배웠다. 어떤 기능이 필요하고 사용자가 어떻게 해야 이 앱을 편하게 사용할지 계속 고민하면서, 화면 흐름을 부드럽게 바꾸고, UI를 친절하게 수정하고, 작은 기술 디테일들을 추가했다.

처음에는 음식 기록과 AI 분석 정도가 중심이었지만, 실제 사용 흐름을 생각하니 로그인, 사용자별 데이터 분리, 목표 이력, 신체 측정 기록, 알림, export/import, Firestore 동기화가 필요해졌다. 기능이 늘어날수록 Repository, DAO, Entity, UI 상태를 분리하는 구조가 중요하다는 것도 체감했다.

또한 Compose UI를 다듬으면서 정보의 우선순위를 정하는 방법을 배웠다. 홈 화면에서는 모든 숫자를 같은 크기로 보여주는 대신 오늘 가장 중요한 칼로리 상태와 매크로 진행 상태를 먼저 보여주도록 바꾸었다. Goals 화면도 곧바로 입력 폼을 보여주기보다 현재 목표를 먼저 보여주는 방식으로 바꾸면서, 사용자 친화적인 앱을 만드는 방법을 배웠다.

5.2 시간 관계상 구현하지 못해 향후 추가하고 싶은 기능

가장 추가하고 싶은 기능은 Add Meal 화면을 더 카메라 친화적인 화면으로 만드는 것이다. 지금은 사용자가 이미지 파일을 선택하고 설명을 입력하는 방식이지만, 향후에는 Add 버튼을 눌렀을 때 카메라 화면이 바로 열리고, 실시간으로 카메라에 보이는 음식이 highlight되도록 만들고 싶다.

촬영 버튼을 누르거나 highlight된 음식을 터치하면 해당 음식만 바로 AI 분석을 시작하고, 분석된 음식 항목을 한 끼 식사 목록에 추가하는 방식이 이상적이다. 이렇게 하면 사용자는 사진 선택, 설명 입력, 분석의 단계를 줄이고, 실제 식사 자리에서 더 빠르게 기록할 수 있다.

그 외에도 Firebase Storage를 이용한 이미지 업로드, Firestore 문서 단위 양방향 동기화, 더 정교한 AI portion estimation, 알림 클릭 시 Add Meal 화면으로 바로 이동, release build signing과 배포 자동화도 향후 발전 방향으로 남아 있다.

6. 프로젝트 요구사항 만족여부 체크리스트

요구사항	만족 여부	구현 내용
4개 이상의 화면 전환	충족	Login, Home, Add Meal, Meal Review, Meal List, Meal Detail, Edit Meal, Goal Settings, Recent Stats, Reminder Settings, Profile 화면을 Navigation Compose로 연결했다.
1개 이상의 리스트 페이지	충족	MealListScreen에서 LazyColumn으로 식단 기록을 표시하고, RecentStatsScreen에서 날짜별 기록과 통계를 요약한다.
이미지 포함	충족	OpenDocument로 음식 이미지를 선택하고, URI 권한을 유지하며, Add/Detail/List 화면에서 표시한다.
데이터베이스 사용	충족	Room DB에 user_accounts, meals, meal_foods, goal_plans, body_measurements를 저장한다.
선택 기능	충족	Firebase AI Gemini 분석, Firebase Auth, Firestore 동기화, 식사 기록 알림, export/import를 구현했다.
최종 산출물	충족	README, 주요 화면 캡처, GIF, HTML 보고서 초안, APK 빌드 산출물을 준비했다.

제출 자료 링크 정리

항목	상태
2분 요약 영상	추후 README에 링크 추가 예정
10분 상세 발표 영상	추후 README에 링크 추가 예정
최종 보고서 PDF	이 HTML 문서를 PDF로 변환한 뒤 업로드 예정
APK 다운로드 링크 및 QR 코드	APK를 Google Drive에 업로드한 뒤 README에 링크와 QR 코드 추가 예정